Assignment #4 — Facade Pattern

Project: Cinema Ticket Booking Facade

Student: BEGALIYEV BAKDAULET **Group:** SE-2526

Course: Software Design Patterns **Language:** Java 17

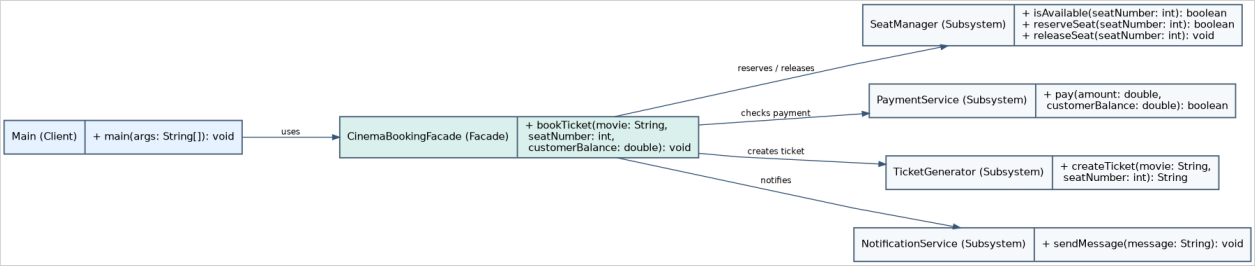
Repository: https://github.com/bakdauletbegaliev/cinematicketbooking

1. Introduction

Booking a cinema ticket requires several tasks: checking the seat, reserving it, validating a simulated payment, generating ticket information, and notifying the customer. Without a facade, the client would need to coordinate these services and handle failures itself. The Facade pattern provides one simple entry point while leaving the individual services separate.

This project uses **CinemaBookingFacade.bookTicket(movie, seatNumber, customerBalance)**. The client calls only this method, while the facade performs the steps in the correct order. If the payment check fails, the previously reserved seat is released (rollback).

2. UML Class Diagram



Reading the diagram: the client uses the Facade; arrows from the Facade point to subsystem classes that it coordinates. A rectangle represents a class; + indicates a public operation; text after : indicates its return type. A more detailed legend is included in docs/UML_Legend.md.

3. UML Legend — responsibilities

Main: Client; starts four demonstration cases and never calls subsystem classes.

CinemaBookingFacade: Facade; organizes seat reservation, payment, ticket creation and messages.

SeatManager: Subsystem; validates seat numbers 1–10, reserves seats, and releases failed reservations.

PaymentService: Subsystem; simulates payment approval when provided funds cover the ticket price.

TicketGenerator: Subsystem; creates readable ticket text.

NotificationService: Subsystem; prints feedback about the booking outcome.

4. Booking Flow and Error Handling

1. Main calls the facade with a movie, seat number and simulated balance.

2. SeatManager tries to reserve the seat; if invalid or unavailable, the facade reports failure.

3. PaymentService checks the price against the balance. If insufficient, the facade calls releaseSeat to roll back.

4. If payment succeeds, TicketGenerator formats a ticket and NotificationService reports success.

```
if (!seatManager.reserveSeat(seatNumber)) {
    notificationService.sendMessage("Seat is unavailable or invalid.");
    return;
}
if (!paymentService.pay(TICKET_PRICE, customerBalance)) {
    seatManager.releaseSeat(seatNumber);
    notificationService.sendMessage("Payment failed. Seat reservation cancelled.");
    return;
}
```

5. Clean Code Principles — five before/after examples

1. Meaningful method names

Before (illustrative): Bad: facade.run(m, n, b);

After (used design): Good: cinema.bookTicket("Interstellar", 3, 5000);

Why: bookTicket communicates the purpose of the operation; descriptive parameters help explain its inputs.

2. Single Responsibility

Before (illustrative): Bad: Main checks seats, verifies payment, and prints a ticket.

After (used design): Good: Main calls cinema.bookTicket(...);

Why: The Facade coordinates; SeatManager, PaymentService, and TicketGenerator each do their own specific job.

3. Small, focused classes

Before (illustrative): Bad: One CinemaSystem class contains all booking, payment and ticket logic.

After (used design): Good: SeatManager.reserveSeat(...); PaymentService.pay(...);

Why: Splitting responsibilities makes each subsystem class easier to understand and adjust.

4. Hide subsystem details

Before (illustrative): Bad: client calls reserveSeat(), pay(), createTicket() separately.

After (used design): Good: cinema.bookTicket(movie, seat, balance);

Why: Clients are not forced to know the sequence of subsystem operations or when rollback is needed.

5. Handle errors and rollback

Before (illustrative): Bad: A failed payment leaves the seat permanently reserved.

After (used design): Good: if (!pay(...)) { releaseSeat(seat); return; }

Why: The facade returns the simulated booking process to a consistent state when payment is declined.

6. Demonstration / Test Cases

Case	Action	Expected result
1	Seat 3, balance 5000	Success + ticket
2	Seat 3 again	Unavailable
3	Seat 5, balance 1000	Payment fails; seat released
4	Seat 5, balance 3000	Success after rollback

7. Limitations

This is a classroom simulation, not a commercial booking system. Seats are stored in memory; a single 10-seat list is shared by the demonstration, not separated by movie or screening. Payment only checks sufficient funds, without charging a real account. Notifications are printed to the terminal. These simplifications keep the Facade pattern easy to understand.

8. Conclusion

The Facade pattern makes booking easier for clients by hiding the order and coordination of several subsystem calls, including the release of a seat after failed payment. The services remain separate, do not reference the facade, and can still be used directly by more advanced clients. Main deliberately uses only the facade.

9. GitHub Repository

https://github.com/bakdauletbegaliev/cinematicketbooking

Important: The link above is the repository address provided by the student; this report does not verify that the code has already been uploaded. This is an Option C free topic requiring instructor approval. The course instructions also require independent work; use this example as a reference, customize it, and be prepared to explain it during the defense.

10. References

Assignment #4 — Facade Pattern (course instructions).

Refactoring.Guru — Facade: https://refactoring.guru/design-patterns/facade